



北京師範大學
BEIJING NORMAL UNIVERSITY

计算机图形学 实验报告

姓名 杨博文 学号 202311081015
院系 人工智能学院 专业 计算机科学与技术

2025 年 3 月 9 日

摘 要

计算机图形学的第一次实验，主要包括 OpenGL 的环境配置、第一个窗口、第一个三角形、上色、上纹理。

目录

1 运行环境及运行方式	3
2 任务 1: 正四面体的渲染	3
2.1 任务要求	3
2.2 关键代码	3
2.3 结果图片	4
3 任务 2: 正四面体的上色	4
3.1 任务要求	4
3.2 关键代码	4
3.3 结果图片	5
4 任务 3: 正四面体的纹理	6
4.1 任务要求	6
4.2 关键代码	6
4.3 结果图片	8
5 写在最后	8

1 运行环境及运行方式

硬件为 Macbook Air, M2 芯片, 操作系统为 MacOS Sequoia 15.0。

IDE 为 Xcode Version16.0。

使用 GLFW+GLAD, 程序在 Xcode 内编译运行。

2 任务 1: 正四面体的渲染

2.1 任务要求

在 hello_triangle.cpp 的基础上, 实现正四面体的渲染。

2.2 关键代码

```
float vertices[] = { // 这里根据计算给出正四面体的顶点坐标
    // 顶点A (后中)
    0.0f, -0.211f, 0.816f,
    // 顶点B (左下)
    -0.5f, -0.5f, 0.0f,
    // 顶点C (右下)
    0.5f, -0.5f, 0.0f,
    // 顶点D (前上)
    0.0f, 0.366f, 0.0f
};

unsigned int indices[] = {
    1, 2, 3,
    0, 1, 2,
    0, 1, 3,
    0, 2, 3
};

...
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
...
glDrawElements(GL_TRIANGLES, 12, GL_UNSIGNED_INT, 0);
```

这里我们通过计算给出正四面体的四个顶点的三维坐标, 我们使用 indices 数组避免记录 12 个坐标, 用 4 个坐标给出 4 个三角形。

2.3 结果图片

开启线框模式后, 输出如图 1 所示。这里我们使用 `glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)` 开启线框模式, 表明我们生成了四个三角形 (虽然这里依旧只能看到三个)。

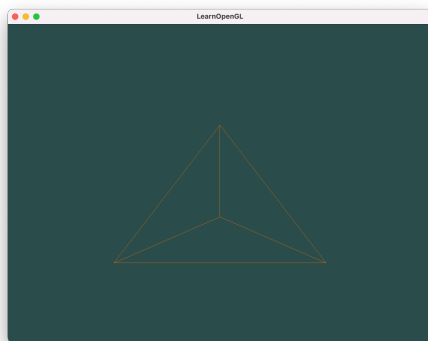


图 1: 实验 1 线框模式输出

关闭线框模式后, 输出如图 2 所示。

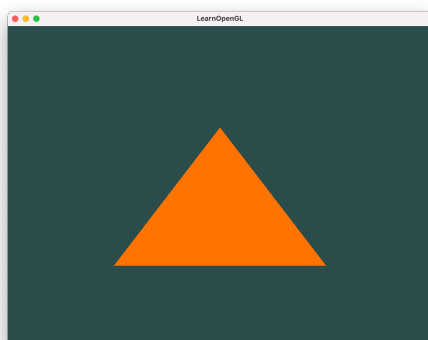


图 2: 实验 1 非线框模式输出

3 任务 2: 正四面体的上色

3.1 任务要求

在任务 1 的基础上, 为不同三角形面设计不同的颜色, 至少一个三角形的三个顶点颜色不同。

3.2 关键代码

```
float vertices[] = {
```

```
// 顶点A (后中)
0.0f, -0.211f, -0.816f, 1.0f, 0.0f, 0.0f,
// 顶点B (左下)
-0.5f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f,
// 顶点C (右下)
0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 1.0f,
// 顶点D (前上)
0.0f, 0.366f, 0.0f, 1.0f, 1.0f, 1.0f,
};
...
// position attribute
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
// color attribute
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float)
, (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);
```

这里，我们设定四个点分别为红、绿、蓝、白四色，使用 RGB 进行设置，然后进行导入。我们的图元是渐变色的。

3.3 结果图片

输出结果如图 3 所示。这里由于我们设定 A 点的 $z < 0$ ，所以红色在视野之中。

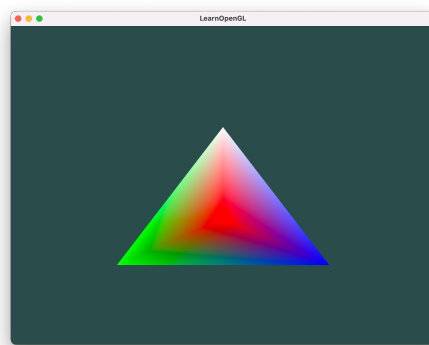


图 3: 实验 2 输出

4 任务 3: 正四面体的纹理

4.1 任务要求

在任务 2 的基础上, 自选纹理图像, 为立方体应用纹理。

4.2 关键代码

```
const char *fragmentShaderSource = "#version 330 core\n"
    "out vec4 FragColor;\n"
    "in vec3 ourColor;\n"
    "in vec2 TexCoord;\n"
    "uniform sampler2D ourTexture;\n"
    "void main()\n"
    "{\n"
    "    vec4 textureColor = texture(ourTexture, TexCoord);\n"
    "    vec4 vertexColor = vec4(ourColor, 1.0);\n"
    "    FragColor = mix(vertexColor, textureColor, 0.5);\n"
    "}\n\n0";

float vertices[] = {
    // position           // color           // texture
    0.0f, -0.211f, -0.816f,  1.0f, 0.0f, 0.0f,  0.5f, 1.0f, // A
    -0.5f, -0.5f, 0.0f,      0.0f, 1.0f, 0.0f,  0.0f, 0.0f, // B
    0.5f, -0.5f, 0.0f,      0.0f, 0.0f, 1.0f,  1.0f, 0.0f, // C
    0.0f, 0.366f, 0.0f,      1.0f, 1.0f, 0.0f,  0.5f, 0.5f  // D
};

...
// position
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);

// color
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float)
, (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);

// texture
```

```
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8 * sizeof(float)
, (void*)(6 * sizeof(float)));
glEnableVertexAttribArray(2);

// load texture
unsigned int texture;
glGenTextures(1, &texture);
glBindTexture(GL_TEXTURE_2D, texture);
...
unsigned char* data = stbi_load("wall.jpg", &width, &height, &nrChannels, 0);
if (data)
{
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB,
    GL_UNSIGNED_BYTE, data);
    glGenerateMipmap(GL_TEXTURE_2D);
}
else
{
    std::cout << "Failed to load texture" << std::endl;
}
...
while (!glfwWindowShouldClose(window))
{
    ...
    glBindTexture(GL_TEXTURE_2D, texture);
    ...
}
```

我们导入纹理的图片，给出每个位置的纹理坐标，贴上纹理即可。我们额外对纹理和颜色进行了合并。

4.3 结果图片

我们给出两个输出结果，第一个是 A 的 $z = -0.816f$ ，这样我们可以看见红色。输出如图 4 所示。

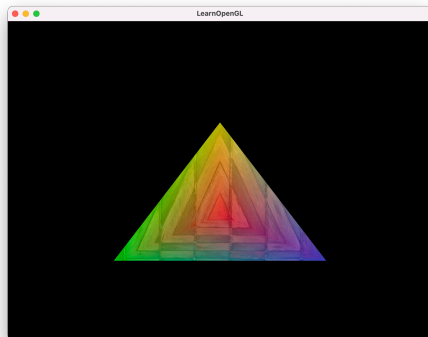


图 4: 实验 3 $z = -0.816f$ 输出

第二个是 A 的 $z = 0.816f$ ，这样我们看不见红色，只能看到一个面。输出如图 5 所示。

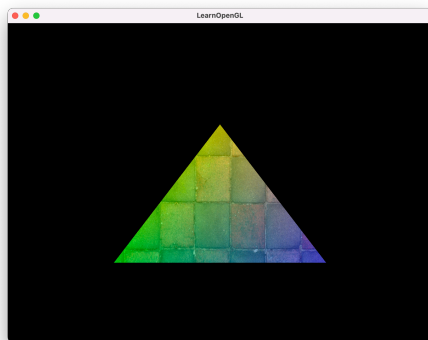


图 5: 实验 3 $z = 0.816f$ 输出

5 写在最后

配环境从周二晚上一直配到了周四晚上，一直没办法解决新建第二个项目的问题，到现在也只能发现和项目命名有关，但具体如何相关暂不知道。从周五晚上开始学习，一直学到现在（周日的 17:00），中间基本没学其他的科目。或许是开始的第一步是最难的，但一开始就看到两百行的啥都看不懂的代码的时候心是死的。报错不知道发生在哪里，中间遇到不同的 z 不影响渲染结果（实际上前后变化）之类的不知所踪的问题。解决也好，不解决也罢。第二周其他科目基本都还没开始实验，就还算有时间能够把周末的两天都给图形学实验，那后续呢？

实验量过大，完全靠自学！以上肺腑，无从夸大，无需夸大。